Adaptive Post-Quantum Secure IPC Framework

Complete Presentation Script, Demo Walkthrough & OS Faculty Viva Guide

GROUP: DEEPANSHU & ADITYA | COURSE: OPERATING SYSTEMS | REPO: ADAPTIVE-PQ-SECURE-IPC | NIST PQC: ML-KEM & ML-DSA

1. Division of Labor: Who Built What?

Keep this crystal clear. If the teacher asks *"What did YOU do versus what did Aditya do?"*, this table and breakdown gives the exact boundary:

Attribute	Deepanshu's Part (The Execution & OS Muscle)	Aditya's Part (The Decision Brain)
Folder & Code	`secure_ipc/` (Python 3, ~1,000 LOC)	`policy_engine/` (C++14, ~4,850 LOC)
Core Question	*"How do we actually enforce security between two OS processes?"*	*"How much security does this specific message need?"*
OS Primitives	**Anonymous Pipes** (`os.pipe`), **Unix Domain Sockets** (`AF_UNIX`), process spawning, nonces, frame serialization.	OS Resource monitoring (CPU load, memory pressure), threat counters, process reputation tracking.
Cryptography	Implemented 5 suites: HMAC-SHA256, X25519, Ed25519, **ML-KEM (Kyber)**, **ML-DSA (Dilithium)**, AES-256-GCM.	Defined the cryptographic suites and built theoretical cost models for latency and memory overhead.
Attack & Defense	Built **anti-replay monotonic counters**, **HKDF re-keying**, and the automated 7-attack test harness.	Designed risk calculation ($R = w_I I + w_L L + w_X I L$), Mosca's quantum inequality check, and safety floors.
Integration	`secure_ipc/policy.py` (JSON bridge) & `framework.py` (runtime loop reacting to attacks).	JSON export format (`aspe decide --json`), 29 unit tests, training dataset calibration.

> **Golden Rule for Speaking:** Whenever speaking about OS processes, pipes, sockets, encryption, signatures, sequence numbers, or attack detections, **Deepanshu answers**. Whenever speaking about how the policy decides, risk formulas, CPU thresholds, or Mosca's inequality, **Aditya answers**.

2. The 60-Second Opening Pitch

Deliver this confidently before typing any command in the terminal:

DEEPANSHU SPEAKS (THE OS PROBLEM):

"Good morning, Sir/Ma'am. In modern Operating Systems, separate processes communicate using IPC mechanisms like Pipes and Unix Domain Sockets. However, these OS primitives are **completely unauthenticated and unencrypted by default**—any program sharing the file descriptor or running locally can intercept or inject data.

Existing security tools apply one static encryption level for everything. But in real systems, encrypting a simple status heartbeat wastes precious CPU cycles, while lightly encrypting an emergency alert leaves it vulnerable. Additionally, future quantum computers will break traditional RSA and ECC cryptography. Our project dynamically assigns the exact required security level per message and enforces it over real OS channels."

ADITYA SPEAKS (THE ARCHITECTURE):

"To solve this, we divided the framework into two cooperating components:
1. **The Decision Brain (C++):** Built by me in `policy_engine/`. It evaluates message criticality, sender trust, and CPU load to calculate risk and select a security level from L1 (lightweight HMAC) to L5 (NIST Post-Quantum Cryptography).
2. **The OS Transport & Execution (Python):** Built by Deepanshu in `secure_ipc/`. It carries out that policy over OS Pipes and Unix Sockets, handling mutual authentication, post-quantum key exchange, anti-replay sequence tracking, and AES-256-GCM encryption."

3. Live Terminal Demo: Step-by-Step Walkthrough

Command 1: Complete End-to-End System Demo

Type in terminal: `make run`

DEEPANSHU SPEAKS (EXPLAINING PART 1 — THE 5 MESSAGES):

"Sir/Ma'am, this command runs our complete integrated framework between two real, independent OS processes: Process A (sender) and Process B (receiver).

In **Part 1**, you see 5 different messages demonstrating dynamic adaptation:

- **Status (L1 BASIC):** Low sensitivity ping. Uses HMAC-SHA-256 with a pre-shared channel key for integrity—no encryption overhead needed.
- **Gaze (L2 CLASSICAL):** Medium sensitivity. Uses classical X25519 key exchange, Ed25519 signatures, and AES-256-GCM. Re-keys every 64 messages.
- **Profile (L3 HYBRID):** High sensitivity. Combines classical X25519 with post-quantum ML-KEM-512 key encapsulation.
- **Emergency (L4 POST_QUANTUM):** Critical data. Fully post-quantum with ML-KEM-768 (Kyber) and ML-DSA-65 (Dilithium).
- **Alert (L5 STRONG_PQ / OPTIMIZED):** Critical data, but simulated under heavy CPU load (80%). The engine selects L5 (ML-KEM-1024 + ML-DSA-87), but intelligently switches to **OPTIMIZED mode**—relaxing re-keying and dropping per-message signatures so the OS does not freeze."

> **DEEPANSHU SPEAKS (EXPLAINING PART 2 — THE ATTACK):**
> "Now look at **Part 2** on the screen. We send six Gaze messages at L2 Classical. At message #3, a simulated Man-In-The-Middle attacker alters the ciphertext in transit.
>
> Process B receives it, detects an **`IntegrityError: authentication tag mismatch`**, rejects the message, and triggers an attack alert."

> **ADITYA SPEAKS (EXPLAINING THE REACTION):**
> "When Process B reports the attack, our framework dynamically halves sender trust from 0.85 to 0.42 and increases the threat level. When message #4 is evaluated, my policy engine automatically escalates Gaze from **L2 Classical to L3 Hybrid (Quantum-Safe)**. Deepanshu's engine automatically renegotiates a new secure session without stopping the processes!"

Command 2: OS Transport Primitives (Pipes vs. Sockets)

Type in terminal: `make demo`

> **DEEPANSHU SPEAKS (YOUR CORE OS PART):**
> "Here, I am demonstrating my secure transport layer operating directly over native POSIX OS primitives: **Anonymous Pipes** and **Unix Domain Sockets** across all 5 security levels."

What to point at on the terminal:

- 👉 **Point at [pipe / L1 BASIC]**: *"Here, Process A and Process B use an **OS Anonymous Pipe (`os.pipe`)**. In Unix, an anonymous pipe works between related parent-child processes via shared file descriptors."*
- 👉 **Point at [socket / L1 BASIC]**: *"Down here, we switch the transport to a **Unix Domain Socket (`AF_UNIX`)**. It uses a filesystem inode path, allowing completely independent, unrelated processes to talk safely."*
- 👉 **Point at 'B: authenticated processA'**: *"Process B authenticates Process A's digital signature against the trusted registry before accepting any payload."*
- 👉 **Point at 'A: got reply ACK'**: *"Process B sends an encrypted acknowledgment back. We derive two independent keys via HKDF ($K_{A o B}$ and $K_{B o A}$) so an attacker cannot reflect A's own messages back to A."*

4. Command 3: The Attack Suite (Explained Like You're 10 Years Old)

Type in terminal: `python3 -m secure_ipc.attacks`

> **THE 10-YEAR-OLD STORY (UNDERSTAND THIS CLEARLY):**
> Imagine you (**Process A**) and your friend (**Process B**) are passing secret letters through a hollow pipe in a wall.
> An evil neighbor (the attacker script) is sitting in the middle of that pipe trying **7 dirty tricks** to fool your friend.
> Command 3 runs a simulation where the hacker tries all 7 tricks against all 5 security levels ($7 imes 5 = 35$ tests).
> The table on your screen is just a **Scoreboard**: **"Detected"** means your friend caught the hacker red-handed and slammed the window shut!

The 7 Hacker Tricks in Plain Words:

Attack Name	What the Hacker Tries (Dirty Trick)	How Your Code Catches It
1. unauthorized process	**The Unknown Stranger:** A random program called `mallory` creates valid keys and knocks on Process B's door.	Process B checks its guest list folder (`keys/`). Mallory is not invited $ o$ Connection slammed shut (`AuthError`).
2. fake process	**The Imposter / ID Theft:** A rogue program connects and lies: *"Hey B, I am Process A!"*, but signs with its own fake key.	Process B checks the signature with the *real* public key of Process A on file $ o$ Fails (`AuthError`).
3. modified message	**Tampering with the Letter:** Process A sends *"Transfer $100"*. The hacker flips one bit in transit to make it *"Transfer $900"*.	We use AES-GCM encryption. It has a cryptographic wax seal (tag). Changing 1 bit shatters the seal $ o$ Dropped (`IntegrityError`).
4. replayed message	**The Photocopy Trick:** The hacker cannot read the secret letter, so he makes a photocopy and sends it a second time.	Every letter has a number: #1, #2, #3... Process B only accepts numbers higher than the last seen $ o$ Dropped instantly (`ReplayError`).
5. invalid authentication	**Smudging the Hello Signature:** The hacker messes up Process A's signature during the first handshake message.	Process B checks the math on the signature. The math fails $ o$ Aborts handshake.
6. corrupted ciphertext	**Injecting Random Garbage:** The hacker blasts pure random garbage bytes down the pipe.	Process B tries to decrypt it. The decryption tag fails $ o$ Caught (`IntegrityError`).
7. invalid signature	**Smudging the Reply Signature:** Process B replies to A, but the hacker messes up B's signature on the way back.	Process A checks B's signature. It fails, so Process A refuses to trust the connection.

🎙️ **What to Point at on the Terminal & Say:**

```
| Attack                | L1 BASIC         | L2 CLASSICAL     | L3 HYBRID        | L4 POST_QUANT
| unauthorized process  | Detected (0.3 ms)| Detected (0.2 ms)| Detected (1.2 ms)| Detected (34.
| modified message      | Detected (0.0 ms)| Detected (0.0 ms)| Detected (0.0 ms)| Detected (0.0
| replayed message      | Detected (0.0 ms)| Detected (0.0 ms)| Detected (0.0 ms)| Detected (0.0
| invalid signature     | Detected (0.3 ms)| Detected (0.5 ms)| Detected (2.9 ms)| Detected (70.
```

SAY THESE 3 SIMPLE POINTS WHILE POINTING:

1. 👉 **Point at 'replayed message' (0.0 ms)** : *"Look at `replayed message`. It is caught in **0.0 milliseconds**. That's because I put a sequence counter (1, 2, 3...) on each packet. Duplicate or old packets are discarded immediately without wasting CPU."*

2. 👉 **Point at 'modified message' (0.0 ms)** : *"Look at `modified message`. Here the test flips a bit in transit. AES-256-GCM authenticated encryption catches it instantly because the authentication tag breaks."*

3. 👉 **Point at 'invalid signature' (0.5 ms at L2 vs 126 ms at L5)** : *"Notice the detection times: At L2 Classical, checking a bad signature takes only **0.5 ms** (standard ECC). At L5 Post-Quantum, it takes **126 ms** because L5 uses NIST's new ML-DSA (Dilithium) lattice-based signature math, which has higher computational overhead."*

5. OS Faculty Viva Questions (Demo-Oriented & Practical)

An OS teacher usually focuses on processes, concurrency, IPC differences, and demo verification rather than deep quantum mathematics. Here are the exact questions to expect:

Q1. Why are you using two separate OS processes instead of two threads?

> **Deepanshu answers:** *"Because threads share the same virtual address space and memory. If one thread is compromised, the attacker already has access to all memory and keys. By using two separate OS processes (Process A and Process B), the OS kernel strictly isolates their memory spaces. Data can only cross the boundary via the IPC channel (pipe or socket), which is where our security layer sits."*

Q2. What is the fundamental OS difference between an Anonymous Pipe and a Unix Domain Socket?

> **Deepanshu answers:** *"An **Anonymous Pipe (`os.pipe`)** is created in kernel memory and is unidirectional; it can only connect related processes (like parent and child) that inherit file descriptors via `fork()`. A **Unix Domain Socket (`AF_UNIX`)** uses a filesystem path as its rendezvous address, allowing completely independent, unrelated processes to initiate bidirectional communication."*

Q3. What happens if you run the demo command a second time? Are keys recreated?

> **Deepanshu answers:** *"The long-term identity keys stored in `secure_ipc/keys/` are preserved and reused so identity persists. However, for every new session, a fresh, random **ephemeral key exchange** is performed, generating brand-new session keys. Even if you run the demo 100 times, the session keys on the wire are always unique."*

Q4. Why is the detection time 0.0 ms for 'replayed message'?

> **Deepanshu answers:** *"Because checking a replay attack does not require expensive cryptographic decryption. Process B maintains an integer counter of the last accepted sequence number. When a packet arrives, it checks `if seq_num <= last_accepted`. This is a single CPU integer comparison, which takes mere nanoseconds, showing up as 0.0 ms."*

Q5. Why did L5 take 126 ms while L2 took 0.5 ms in Command 3?

> **Deepanshu answers:** *"Level 2 uses classical Ed25519 elliptic-curve signatures, which are tiny (64 bytes) and heavily optimized in C. Level 5 uses NIST Post-Quantum ML-DSA-87 (Dilithium), where signatures are over 4,600 bytes and involve complex high-dimensional polynomial lattice math, currently running in pure Python."*

Q6. How does Process B know that Process A is not an imposter?

> **Deepanshu answers:** *"During the initial handshake, Process A generates a random nonce, attaches its name and security profile, and signs the entire package with its private key. Process B looks up Process A's known public key in the trusted registry folder and verifies the signature. An imposter cannot produce that signature without Process A's private key."*

Q7. What is Re-keying and why is it important in Operating Systems?

> **Deepanshu answers:** *"Re-keying limits the 'blast radius' if a process is breached. After a set number of messages (e.g. every 64 messages at L2, or every 16 at L4), both processes derive a new session key using HKDF and delete the old key from memory. If malware later dumps process RAM, it cannot decrypt past messages (forward secrecy) or future messages."*

Q8. How does Aditya's C++ program communicate with your Python program?

> **Deepanshu / Aditya answer:** *"Through an inter-language JSON IPC interface. In `secure_ipc/policy.py`, Python executes Aditya's compiled binary (`aspe decide --json`) passing the message context. The C++ engine returns the policy decision (Level, Mode, Re-key interval) as a standard JSON string in just 2-3 milliseconds."*

Q9. Deepanshu, summarize in 30 seconds what YOUR individual contribution was in this project.

> **Deepanshu answers:** *"I built the entire `secure_ipc/` subsystem in Python. I implemented inter-process communication over OS Pipes and Unix Domain Sockets, implemented all 5 cryptographic suites including NIST post-quantum ML-KEM and ML-DSA, engineered mutual process authentication, designed the anti-replay sequence numbering, wrote the automated 7-attack test suite, and built the runtime feedback loop that escalates security upon detecting an attack."*
